



Cartera — Handoff Nuxt 4

App mobile-first de digitalización de tickets y finanzas personales. Este documento es la referencia técnica completa para implementar el prototipo en producción con **Nuxt 4**.

- Alta Fidelidad
- Nuxt 4
- Vue 3 Composition API
- Pinia
- Mobile-first PWA

Overview

Cartera permite a los usuarios fotografiar tickets físicos (con extracción automática por IA), añadirlos manualmente, consultar estadísticas de gasto por periodo/categoría y gestionar su perfil. Diseñada como PWA mobile-first con soporte offline.

Pantallas principales

- Login / Registro
- Dashboard con gráficos
- Lista de tickets
- Detalle del ticket
- Escaneo IA / Formulario manual
- Estadísticas
- Perfil + subpantallas

Funcionalidades clave

- OCR + extracción IA (OpenAI Vision)
- Categorización automática
- Gráficos día/semana/mes/año
- Descarga de tickets como imagen
- Auth con email + OAuth (Google/Apple)
- PWA con soporte offline

Tech Stack

CAPA	TECNOLOGÍA	MOTIVO
nuxt@4	Nuxt 4	Framework principal, SSR + PWA
@pinia/nuxt	Pinia	Estado global (user, tickets, stats)
@nuxt/ui	NuxtUI v3	Componentes headless accesibles
tailwindcss	Tailwind CSS 4	Utilidades + CSS variables Dracula
vee-validate + zod	VeeValidate + Zod	Validación de formularios tipada
nuxt-auth-utils	nuxt-auth-utils	Auth sessions + OAuth providers

@vite-pwa/nuxt	VitePWA	Service worker + instalación offline
openai	OpenAI SDK	Extracción IA de tickets (Vision API)
@supabase/supabase-js	Supabase	Base de datos + Storage para imágenes
chart.js / vue-chartjs	vue-chartjs	Gráficos de estadísticas

Estructura de archivos

```
cartera/
├─ app/                                # Nuxt 4 app directory
│   └─ pages/
│       ├── index.vue                 # Redirect → /dashboard o /login
│       ├── login.vue                 # Pantalla de login/registro
│       └─ dashboard/
│           ├── index.vue             # Dashboard principal
│           └─ tickets/
│               ├── index.vue         # Lista de tickets
│               ├── nuevo.vue         # Modal selección: IA / Manual
│               ├── escanear.vue      # Flujo OCR + extracción IA
│               ├── manual.vue        # Formulario manual
│               └─ [id].vue           # Detalle del ticket
│           └─ estadisticas/
│               └─ index.vue          # Pantalla de estadísticas
│       └─ perfil/
│           ├── index.vue             # Perfil principal
│           ├── datos.vue             # Editar datos personales
│           ├── seguridad.vue         # Seguridad y privacidad
│           ├── pagos.vue             # Métodos de pago
│           └─ ia.vue                 # Preferencias IA
│
│   └─ components/
│       └─ ui/                        # Componentes base
│           ├── AppButton.vue
│           ├── AppInput.vue
│           ├── AppCard.vue
│           ├── AppBadge.vue
│           └─ AppBottomSheet.vue
│       └─ layout/
│           ├── BottomNav.vue
│           └─ MobileHeader.vue
│       └─ tickets/
│           ├── TicketRow.vue
│           └─ TicketDetail.vue
```

```

├── CategoryIllustration.vue
├── ReceiptPreview.vue
├── AIScanProgress.vue
├── stats/
│   ├── PeriodSelector.vue
│   ├── BarChart.vue
│   ├── LineChart.vue
│   ├── DonutChart.vue
│   ├── CategoryBars.vue
│   └── StatsFiltersSheet.vue
├── dashboard/
│   ├── BalanceCard.vue
│   └── WeeklyChart.vue
├── composables/
│   ├── useTickets.ts
│   ├── useAIExtraction.ts
│   ├── useStats.ts
│   ├── useUser.ts
│   └── useDownload.ts
├── stores/
│   ├── auth.ts
│   ├── tickets.ts
│   └── user.ts
├── middleware/
│   └── auth.ts # Redirect si no hay sesión
├── types/
│   └── index.ts # Interfaces globales
├── server/
│   └── api/
│       ├── tickets/ # CRUD tickets
│       ├── ai/extract.post.ts # OCR + extracción IA
│       └── auth/ # Login / OAuth
├── nuxt.config.ts
├── tailwind.config.ts
└── app.config.ts

```





tailwind.config.ts

```
export default defineConfig({
  theme: {
    extend: {
      colors: {
        dracula: {
          bg:      '#282a36',
          card:    '#44475a',
          card2:   '#383a4a',
          text:    '#f8f8f2',
          muted:   '#6272a4',
          purple:  '#bd93f9',
          pink:    '#ff79c6',
          green:   '#50fa7b',
          cyan:    '#8be9fd',
          yellow:  '#f1fa8c',
          orange:  '#ffb86c',
          red:     '#ff5555',
        }
      },
    },
    fontFamily: {
      sans: ['Inter', 'system-ui', 'sans-serif'],
      mono: ['Fira Code', 'monospace'],
    },
    borderRadius: {
      '2xl': '16px',
      '3xl': '20px',
      '4xl': '24px',
    }
  }
})
```

Espaciado y tipografía

TOKEN	VALOR	USO
text-xs	10–11px	Labels, badges, meta
text-sm	12–13px	Subtítulos, descripciones
text-base	14–15px	Cuerpo, listas
text-lg	17–18px	Títulos de pantalla


```
// app/middleware/auth.ts
export default defineNuxtRouteMiddleware((to) => {
  const { loggedIn } = useUserSession()
  const publicRoutes = ['/login']

  if (!loggedIn.value && !publicRoutes.includes(to.path)) {
    return navigateTo('/login')
  }
})
```

Componentes clave

TicketRow.vue

[tickets/](#)

```
// Props
interface Props {
  ticket: Ticket
  compact?: boolean // modo compacto para dashboard
}

// Emits
const emit = defineEmits<{
  click: [ticket: Ticket]
}>()
```

CategoryIllustration.vue

[tickets/](#)

```
// Props – renderiza SVG ilustrado por categoría
interface Props {
  categoria: TicketCategoria // 'Alimentación' | 'Transporte' | etc.
  size?: number // default: 90
}

// Categorías con ilustración: Alimentación, Transporte, Ropa,
// Suscripciones, Salud, Restaurantes, Hogar, Ocio
```

AIScanProgress.vue

[tickets/](#)

```
// 3 fases: 'scan' | 'loading' | 'result'
interface Props {
  phase: 'scan' | 'loading' | 'result'
  extractedData?: ExtractedTicket
  progress?: number // 0-100
}
const emit = defineEmits<{
  upload: [file: File]
  confirm: [data: ExtractedTicket]
  editManual: []
}>()
```

BottomNav.vue

layout/

```
// Nav fija inferior con FAB central (+)
// Tabs: dashboard | tickets | [fab] | estadisticas | perfil
const route = useRoute()
const activeTab = computed(() => {
  if (route.path.startsWith('/tickets')) return 'tickets'
  if (route.path.startsWith('/estadisticas')) return 'estadisticas'
  if (route.path.startsWith('/perfil')) return 'perfil'
  return 'dashboard'
})
```

DonutChart.vue / BarChart.vue / LineChart.vue

stats/

```
// Usar vue-chartjs con Chart.js – configuración dark theme
// DonutChart: datos por categoría
interface ChartData {
  label: string
  value: number
  color: string
}
interface Props {
  data: ChartData[]
  size?: number
}

// BarChart / LineChart: serie temporal
interface Props {
  points: number[]
  labels: string[]
}
```

```
color?: string // default: '#bd93f9'
type: 'bar' | 'line'
}
```

⚡ Composables

useTickets()

CRUD completo de tickets. Fetching con `useFetch`, caché automática y optimistic updates.

```
export function useTickets() {
  const store = useTicketsStore()

  const { data: tickets, pending, refresh } = useFetch<Ticket[]>('/api/tickets', {
    default: () => [],
  })

  const createTicket = async (data: CreateTicketDto) => {
    const ticket = await $fetch('/api/tickets', {
      method: 'POST', body: data
    })
    await refresh()
    return ticket
  }

  const deleteTicket = async (id: string) => {
    await $fetch(`/api/tickets/${id}`, { method: 'DELETE' })
    await refresh()
  }

  return { tickets, pending, createTicket, deleteTicket, refresh }
}
```

useAIExtraction()

Gestiona el flujo de extracción IA: upload de imagen → llamada a OpenAI Vision → parsing de JSON.

```
export function useAIExtraction() {
  const phase = ref<'idle' | 'uploading' | 'extracting' | 'done' | 'error'>('idle')
  const progress = ref(0)
  const result = ref<ExtractedTicket | null>(null)
```



```

const extract = async (file: File) => {
  phase.value = 'uploading'
  const formData = new FormData()
  formData.append('image', file)

  phase.value = 'extracting'
  // Simula progreso mientras espera la IA
  const interval = setInterval(() => {
    if (progress.value < 90) progress.value += 8
  }, 200)

  try {
    result.value = await $fetch('/api/ai/extract', {
      method: 'POST', body: formData
    })
    progress.value = 100
    phase.value = 'done'
  } catch {
    phase.value = 'error'
  } finally {
    clearInterval(interval)
  }
}

return { phase, progress, result, extract }
}

```

useStats()

Agrega tickets por periodo y categoría para las visualizaciones.

```

export function useStats(period: Ref<'dia' | 'semana' | 'mes' | 'año'>) {
  const { tickets } = useTickets()

  const filtered = computed(() =>
    tickets.value.filter(t => isInPeriod(t.fecha, period.value))
  )

  const total = computed(() =>
    filtered.value.reduce((s, t) => s + t.total, 0)
  )

  const byCategory = computed(() =>
    groupBy(filtered.value, 'categoria')
  )
}

```

```

const chartPoints = computed(() =>
  buildTimeSeries(filtered.value, period.value)
)

return { total, byCategory, chartPoints, filtered }
}

```

useDownload()

Genera y descarga el ticket como SVG o PDF desde el cliente.

```

export function useDownload() {
  const downloadTicketSVG = (ticket: Ticket, el: HTMLElement) => {
    const data = new XMLSerializer().serializeToString(el)
    const blob = new Blob([data], { type: 'image/svg+xml' })
    const url = URL.createObjectURL(blob)
    const a = document.createElement('a')
    a.href = url
    a.download = `ticket-${ticket.comercio}-${ticket.fecha}.svg`
    a.click()
    URL.revokeObjectURL(url)
  }
  return { downloadTicketSVG }
}

```

Pinia Stores

stores/auth.ts

```

export const useAuthStore = defineStore('auth', () => {
  const { loggedIn, user, clear } = useUserSession()

  const login = async (email: string, password: string) => {
    await $fetch('/api/auth/login', { method: 'POST', body: { email, password } })
  }

  const logout = async () => {
    await $fetch('/api/auth/logout', { method: 'POST' })
    await clear()
    await navigateTo('/login')
  }
}

```

```

    }

    return { loggedIn, user, login, logout }
  })

```

stores/tickets.ts

```

export const useTicketsStore = defineStore('tickets', () => {
  const tickets = ref<Ticket[]>([])
  const activeFilter = ref<TicketCategoria | null>(null)

  const filtered = computed(() =>
    activeFilter.value
      ? tickets.value.filter(t => t.categoria === activeFilter.value)
      : tickets.value
  )

  return { tickets, activeFilter, filtered }
})

```



Types / Interfaces

```

// app/types/index.ts

export type TicketCategoria =
  | 'Alimentación' | 'Transporte' | 'Ropa'
  | 'Restaurantes' | 'Suscripciones' | 'Salud'
  | 'Hogar' | 'Ocio' | 'Tecnología' | 'Otro'

export interface Ticket {
  id: string
  userId: string
  comercio: string
  fecha: string // ISO 8601
  total: number
  iva?: number
  categoria: TicketCategoria
  metodoPago?: string
  notas?: string
  imageUrl?: string // URL Supabase Storage

```

```
items?: TicketItem[]
extractedByAI: boolean
aiConfidence?: number // 0-1
createdAt: string
}

export interface TicketItem {
  nombre: string
  precio: number
  cantidad?: number
}

export interface ExtractedTicket {
  comercio: string
  fecha: string
  total: number
  iva?: number
  items: TicketItem[]
  metodoPago?: string
  categoria: TicketCategoria
  confianza: number
}

export interface CreateTicketDto extends Omit<Ticket, 'id' | 'userId' | 'createdAt'> {}

export interface UserProfile {
  id: string
  nombre: string
  email: string
  telefono?: string
  ciudad?: string
  divisa: 'EUR' | 'USD' | 'GBP' | 'JPY' | 'MXN'
  avatarUrl?: string
  createdAt: string
}

export type StatsPeriod = 'dia' | 'semana' | 'mes' | 'año'

export interface CategoryStat {
  categoria: TicketCategoria
  amount: number
  pct: number
  color: string
}
```

API Endpoints (server/api/)

MÉTODO + RUTA	ARCHIVO	BODY / PARAMS	RESPUESTA
POST /api/auth/login	auth/login.post.ts	{ email, password }	session
POST /api/auth/logout	auth/logout.post.ts	—	200 OK
GET /api/tickets	tickets/index.get.ts	?period, ?categoria	Ticket[]
POST /api/tickets	tickets/index.post.ts	CreateTicketDto	Ticket
GET /api/tickets/:id	tickets/[id].get.ts	id: string	Ticket
PUT /api/tickets/:id	tickets/[id].put.ts	Partial<Ticket>	Ticket
DELETE /api/tickets/:id	tickets/[id].delete.ts	id: string	204
POST /api/ai/extract	ai/extract.post.ts	FormData (image)	ExtractedTicket
GET /api/stats	stats/index.get.ts	?period	StatsResponse
GET /api/user	user/index.get.ts	—	UserProfile
PUT /api/user	user/index.put.ts	Partial<UserProfile>	UserProfile

Extracción IA — server handler

```
// server/api/ai/extract.post.ts
import OpenAI from 'openai'

export default defineEventHandler(async (event) => {
  const formData = await readFormData(event)
  const image = formData.get('image') as File
  const base64 = await fileToBase64(image)

  const openai = new OpenAI({ apiKey: useRuntimeConfig().openaiKey })

  const response = await openai.chat.completions.create({
    model: 'gpt-4o',
    messages: [{
      role: 'user',
      content: [
        { type: 'text', text: `Extrae los datos de este ticket de compra como JSON con:
          { comercio, fecha (ISO), total, iva, items:[{nombre,precio}],
            metodoPago, categoria, confianza (0-1) }` },
        { type: 'image_url', image_url: { url: `data:image/jpeg;base64,${base64}` } }
      ]
    }],
    response_format: { type: 'json_object' },
  })
})
```

```
})
```

```
return JSON.parse(response.choices[0].message.content!)
})
```



Todas las pantallas

/login

Login

Email/password + OAuth Google/Apple. Enlace a recuperar contraseña.

- Logo + nombre app centrado
- Inputs email y password con iconos
- Botón primario degradado purple → pink
- Divider "o continúa con" + botones OAuth

/dashboard

Dashboard

Visión general: saldo, gráficos semanales, últimos 4 tickets y CTA de escaneo.

- Saludo + avatar (→ /perfil)
- Card de saldo con ingresos/gastos
- Grid: bar chart semanal + donut por categoría
- Lista últimos tickets (clickable → detalle)

/tickets

Lista Tickets

Todos los tickets con filtros por categoría.

- Pills de filtro (scrollable)
- Lista de TicketRow con categoría pill
- Tap → /tickets/:id

/tickets/:id

Detalle Ticket

Hero con ilustración SVG de categoría. Tabs: Detalles / Imagen ticket.

- Ilustración SVG según categoría
- Badge "Verificado por IA"
- Todos los campos + líneas de productos
- Previsualización SVG del ticket
- Botón descargar como SVG

/tickets/escanear

Escanear IA

3 fases: scan → loading (scan-line animado) → result (JSON animado).

- Visor de cámara con guías de esquina
- Upload zone con hover state
- Animación scan-line verde sobre ticket
- JSON revelado línea a línea

/tickets/manual

Formulario Manual

Formulario completo con validación Zod.

- Campos: comercio, fecha, total, categoría, método, notas
- Inputs con iconos y labels uppercase
- Botón desactivado hasta campos requeridos

/estadisticas

Estadísticas

/perfil

Perfil

Gráficos y análisis por periodo con filtros avanzados.

- Period selector: Día/Semana/Mes/Año
- Chart barras o línea con toggle
- Donut + barras horizontales por categoría
- Top comercios
- Filtros: agrupar, ordenar, comparar

Menú de secciones + cerrar sesión con confirmación modal.

- Avatar con iniciales (editable)
- Stats: tickets, gastado, meses
- 4 secciones navegables
- Logout con modal de confirmación

Flujo de navegación

Login

`/login` → `onLogin()` → `/dashboard`

Tab bar (siempre visible en rutas privadas)

 Inicio → `/dashboard`

 Tickets → `/tickets`

+ FAB → bottom sheet → `/tickets/escanear` | `/tickets/manual`

 Estadísticas → `/estadisticas`

 Perfil → `/perfil`

Dashboard

Avatar (arriba derecha) → `/perfil`

TicketRow tap → `/tickets/:id`

Banner "Escanea" → bottom sheet

Tickets

TicketRow tap → `/tickets/:id`

Ticket Detail

Tab "Detalles" → campos + productos

Tab "Imagen" → SVG preview + descargar

← back → `/tickets`

Perfil

Datos personales → `/perfil/datos` (guardar → vuelve a `/perfil`)

Seguridad → `/perfil/seguridad`

Métodos de pago → `/perfil/pagos`

Preferencias IA → `/perfil/ia`

Cerrar sesión → modal confirm → `logout()` → `/login`

El prototipo de referencia completo se encuentra en [Cartera.html](#) — ábrelo para ver todas las interacciones, animaciones y estados.

